Improving the Swiss Pairing System

from my Master's thesis

Wissam Nael Pheng

Supervisor : Amire Bendjeddou



Mathematics Section (SMA)
School of Basic Sciences (SB)
Spring 2024

Contents

1 Acknowledgements **2**

2 Introduction **3**

3 The Swiss Pairing System **5**
3.1 History 5
3.2 Overview of the different variations of the Swiss Pairing System in chess 6

4 Critics of the Swiss Dutch Pairing System and goals **8**

5 Related literature **10**

6 Modelization of a tournament **11**
6.1 Existence of an edge 12
6.2 Weights of the edges 13
6.3 Maximum weight matching 14

7 Set up for simulations and tests **18**
7.1 Elo rating vs playing strength 18
7.2 Number of players and rounds 19
7.3 Assigning colors 20
7.4 Metrics to evaluate a system of tournament 20

8 Evaluation of the base systems **23**
8.1 General ranking 23
8.2 Top k 25
8.3 Subcategories 27
8.4 Ideal number of rounds 28
8.5 Oscillating behaviour of players across rounds 30

9 Introducing the Pheng System **31**
9.1 General ranking 32
9.2 Top k and Subcategories 34
9.3 Effect of seeding 36

10 Generalizing using an AI approach : the Neural Pairing System **40**

11 Conclusion **42**

1 Acknowledgements

I would like to deeply thank Amire Benjeddou who supervised my project showing great support all the semester. I am extremely greatful to Clément Hongler who gave me the chance to work on this amazing project, which allowed me to combine my favorite topics of my studies with my passion.

I would also like to thank my friends who motivated me even more that I already was in completing this exciting project, and my family for their unconditional support.

2 Introduction

In various events involving selection or competition, tournament design is a crucial step to ensure its success. The goals can be different from one tournament to another : finding the best participant, qualifying the best ones to another competition, letting everyone enjoy the event, maintaining tension and suspense about the results until the very last round, with a breathtaking finish, or a combination of all of these. In this project we set a focus on chess tournaments and the optimization of ranking quality, which is the ability of one system to successfully rank all the players.

The Swiss pairing system is particularly used in chess and allows to establish a full ranking focusing on finding the winner in around $log_2(n)$ rounds where n is the number of players. After this number of rounds, no two players have the full number of points. There is no elimination and all the participants play the same number of rounds, just like in the round-robin format (but in this one they play many more rounds). In sports where all the games of a round can be played simultaneously without additional constraints, this system is an interesting choice ; this is the case for chess, but in sports like tennis or football it is not always obvious whether making everyone play a game is relevant.

This format in chess is used at various levels : from 1-day local tournaments to 1-week international opens, national and continental championships, Olympiads, and many others. "Improving it significantly" has various definitions according to the context and the goals of the event, but it can have an impact on the way these tournaments are organized.

	Knockout	Round-Robin	Swiss
Participants play the same number of rounds	No	Yes	Yes
Participants play around log(n) rounds	Yes	No	Yes

Figure 1: The Swiss System is very popular to organize tournaments open to many participants (n is the number of participants)

In the Swiss system, players are all paired together at each round. In general the pairs are done between participants with the same number of points (as long as it is possible). As there is not always exactly 2^r participants where r is the number of rounds, and as a draw is also an option apart from

winning and losing, it is possible that several players are tied for first place at the end. Defining appropriate tie-breaking rules is key as there are almost always people with the same number of points. A set of participants with the exact same number of points, independently from their tie-breaks, is called a *scoregroup*. The *top scoregroup* and the *last scoregroup* are respectively the set of participants with the greatest and smallest number of points. The tie breaking rules are used for most of the ranking within scoregroups, but some additional tie-breaking short games may be played for the top scoregroup, depending on the rules of the tournament. We will consider in this project that no such match will be played, and the first place will be decided as any other (by the tie-breaks).

Another criteria to take into account when pairing in chess and other sports is asymmetry ; we mentioned football where one side may have the advantage of playing at home in certain competitions, and in chess the importance of colors. In chess, the player with the White pieces moves first, giving a very slight advantage but most importantly breaking the apparent symmetry of the initial set up of the chessboard. As every confirmed chess player knows, playing with one color or the other is entirely different when it comes to studying the theory of the game. Stats show that in practice, White tends to have more success especially at high level [24d]. It is then important to ensure that all the players of the tournaments have - as far as possible - the same number of times the White and Black pieces.

Here is a general algorithm describing the idea of the Swiss system without entering into details :

1. Establish the initial ranking of players (arbitrary or with a seeding method)

2. Pair all the players with the same number of points as long as possible (at the beginning they all have 0). To decide who to pair within a scoregroup, aim for the best balance of colors (number of times each player has White and Black pieces), and if some choices remain, follow a strategy depending on the format (Dutch, Burstein, ... detailed later). Two players cannot play if they have already played together.

3. Let the players play their matches, and record the results. In case of odd number of players, one doesn't play and get a free point instead

4. Return to step 2 until the right number of rounds is played (this number is decided in advance by the organizer)

5. Establish the final ranking : rank the players by points, then by every Tie-Breaker according to the specific rules of the tournament

There are many Tie-breaking options [24c]:

- The Sonneborn-Berger of a player is the sum of all defeated opponents' points to which we add half the sum of all opponents the player drew with. This is mostly used in round-robin tournaments.
- The Bucholz is simply the sum of all opponents' scores. It evaluates how well the opposition did, in other words how tough were the pairings of the player.
- The Bucholz Cut-k is the Bucholz but without the term corresponding to the k opponents with the lowest score. For example, if the opponents have scores 4, 2 and 5, the Bucholz Cut-1 is 4+5=9. k = 0, 1 are the most used values (0 being the regular Bucholz), with Bucholz Cut-1 often being the main tie-break and Bucholz a secondary one.
- There are many other tie-breaks such as Rating Performance, Cumulative, Game Points for teams, ...

In the following, we will try to satisfy a correct fairness in terms of color assignments, but mostly work on improving the ranking quality. For this, we define several goals and metrics, then study the system more deeply to come up with a solution to the possible flaws.

3 The Swiss Pairing System

3.1 History

The Swiss system was invented in Switzerland, as the name suggests. Its creator, Dr Julius Müller (1857-1917) was a teacher in Brugg (canton of Argau). He reportedly tested his pairing system in the First Swiss Chess Tournament in 1889, but the official date comes years of improvements after, on June 15th 1895 corresponding to the Fifth Swiss Tournament.

The Swiss System is now the most used system for chess, but it is also used in esports, go, scrabble, debates competitions, card games, badminton and many others. The new system used for the UEFA Champions League, Europa

League and Conference League from 2024 will also share similarities with the Swiss System. However, as the pairings will be determined in advance to let the clubs organize their schedules, it is not a Swiss System (where all games of a round have to be completed to determine the next pairings).

But the Swiss system is not only used in tournaments or contests. For example, it has been used in AI-based methods for person identification [Wei+15]. It can be a powerful tool in situations where pairwise comparisons are needed to establish a ranking.

3.2 Overview of the different variations of the Swiss Pairing System in chess

As described previously, the Swiss System aims to pair participants of similar level together in order to determine the best. There are several ways to do so. The first priority is to pair people with the same number of points (initially, they all have zero points, except in "Accelerated Systems"). So we regroup participants into scoregroups, and have to decide how to pair players with the same number of points. We rank the players according to their **Elo**. Let's assume that there are 10 players in a scoregroup, ranked by Elo from 1 to 10 (1 being the strongest). The **Burstein system** consists in pairing 1 with 10, 2 with 9, etc. By maximizing the difference of levels, the favorites get an advantage and may have more chances to score well. A downside is the obvious inequality of treatment with respect to players' Elo. Several decisions can be taken in the case one of these pairings in forbidden, typically if some paired players have already played together. One possibility is to permute players within the scoregroup. It is also possible to exclude players from the scoregroup and include them in another one. This system is recognized by FIDE [24b] and can officially be used in international tournaments.

The **Monrad system** is very uncommon worldwide and mostly used in Nordic countries. The idea is to pair players who are close in the ranking : 1-2, 3-4, etc. This makes players of similar level (in particular the strongest ones) meet each other right at the beginning.

The **Dutch system** is the most commonly used ; it consists in pairing players from the first half with players from the second half, in order to harmonize the differences of strengths. In our example, we would get 1-6, 2-7, 3-8, 4-9, 5-10. Although this makes strength differences equal, in practice they remain high at the very beginning of a tournament. This system is the

most popular and recognized by FIDE.

An idea proposed many times in the past is to simply pair randomly within a scoregroup. This would avoid players with low Elos to be forced to face a stronger opposition than those with high Elos at the beginning. The **Swiss Random System** however may pair the strongest players together in the first rounds of the tournament (as it happened in 2017 Isle Of Man's Open with 2nd and 3rd seeds paired at round 1), which would eliminate some of them from the title race very early and create very heterogeneous paths for all participants. Sauer et al. [PL23] shows how the Swiss Random & Monrad perform weaker than other systems. In tournament without seeding (Elo), the Dutch and Burstein are equivalent to the Swiss Random.

Another pairing system proposed by Sauer et al. [PL23] is the **Swiss Random2**, combining the ideas of the Dutch and Swiss Random : players from the 1st half of a scoregroup are paired randomly with players with the 2nd half. As well as the Swiss Random, this system is not recognized by the International Chess Federation.

There are of course many other pairings systems, like the Dubov system recognized by FIDE and that aims to harmonize the Average Rating Opposition (ARO) in scoregroups, so that two players with the same number of points have faced a similar opposition. The general efficiency of the system is comparable to the Dutch system as the goal is partially reached for middle scoregroups, but slightly less for top and bottom scoregroups (M. Held, 2020 [Hel20]).

We design here a new Swiss Pairing System that, thanks to a study of these systems, allows us to propose an original solution.

4 Critics of the Swiss Dutch Pairing System and goals

Even though the Dutch Pairing System is the most commonly used around the world for chess, it presents more or less clear fallacies. Because experienced players have participated for years under this format, any issue with the final output is generally well accepted.

Firstly, the Dutch system as well as most of the other Swiss Systems aims to find the best player of the tournament. The middle of the ranking will be somehow ranked by strength, but this is not the main goal of the system. By symmetry, the last place is also well determined in case of even number of players. In case of an odd number of players, it is more uncertain as it is one of the least successful players who generally receive the bye, which usually rewards a nonzero number of points.



Figure 2: Extract of the ranking of the 2023 French Students Chess championships (taken on the French Chess Federation website [23]) between ranks 24 and 31 included. The first column is the Elo. Players marked with an F in the second column (JunF, SenF, SenF) are three female players competing for the national title and to represent France at the World Students Championship. The Third column (flags excluded) is the number of points, the three last ones are tie-breaks. As these values show, the last of the three women had a much tougher opposition due to occupying the first spots for a long time, but failed by a 0.5pt difference. This is an example of when the Swiss (Dutch) system fails to determine the best ranking as it is focused on the 1st overall rank and not the other categories where crucial titles can be won.

One of the points where fallacies appear clearly is at the border of score-

groups (see figure 2). The first player of a scoregroup, ranked by an appropriate Tie breaker, has generally faced a much stronger opposition than the last player of the upper scoregroup. Calculations over 10.000 simulated tournaments (32 players, 7 rounds, Dutch System) show that 70% of such scoregroups present a difference of at least 1 point of Bucholz Cut-1 in the wrong way (the player with less points has faced a much stronger opposition). This number climbs up to 81% if we increase the size of the event (100 players, 9 rounds, Dutch System). It is of course not suitable to "manually" switch the two players at the border, as the system is supposed to rank according to points first, then Tie Breaks. One could design a function that takes into account both points and Tie Breaks, then rank according to this value, but in practice it is appreciated that the value determining the ranking is easy to calculate in order to understand quickly the stakes at the last round(s). Bucholz alone cannot rank the players as they would have to lose on purpose their last game to increase it by 1 compared to the scenario where they win, as their last opponent would get one more point. There are approaches such as the Keizer system that rewards more points if the opponent is stronger (according to current ranking for the Keizer system, or Elo in some propositions of systems), so that the difficulty of the opposition has a direct impact on the main ranking criteria (points). However the Elo system is far from being an accurate representation of a player's strength at a tournament, and the presence of unrated players may cause trouble.

Another issue, which is in fact a cause to the previous one, is the importance of the last round. Although previous works have shown that such strategies are not always good to adopt [CFL23], players can lose the first rounds in order to have easier pairings then and finish with a great rank. This well-known phenomenon is called the Swiss Gambit, that Tie Breaker rules aim to limit, but cannot : the number of points remain the main ranking criteria, which yields the previously mentioned problem. Moreover, the last round can make a participant fall from 1st place (or any) to very far in the ranking. It often determines if the player makes a good or bad performance. We will discuss such mechanisms in part 8.5. We will then define a system that takes into account the logic of adding more information to the main ranking criteria.

There are other problems due to the way the Swiss System is used. Most serious competitions give money prizes to the best players. Some tournaments rewards spots to national or continental championships. In particular, the European Individual Championship qualify the 23 (in 2023) best players

for the World Cup. However, this system is only reliable for the very first places. It is then necessary to make it accurate for a certain number of spots instead of the first only.

A lot of competitions, especially open events at amateur level, also reward qualifying spots or money prizes for specific categories of participants : best young players under 18, best women, best players from the country, from a certain category of Elo rating, and others. As we will show later, the results are again very poor as there is a lot of randomness at ranking average players with the Dutch system.

Later we propose AI-driven systems to tackle these issues with appropriate modelling and fine-tuning. For now we define three goals :

1. Improving the overall ranking
2. For some integer k, focus on ranking correctly the top k players
3. For some categories c = 1, ..., C and integers k_1, ..., k_C, focus on ranking correctly the top k_c players within category c

Achieving goal 1 will probably help improving the two others. Solving goal 2 or 3 are generalizations of the case where we only try to find the best player. For all of these goals, the metrics we use will bring more details ; it is interesting to consider several ones to adapt to the rules of every tournament. In practice, we will tackle the third problem by considering $C = 1$ for simplicity.

5 Related literature

Although systems of tournaments such as knock-out and round-robin have been studied and have many applications, the Swiss system have only been more discussed in the last years. Appleton [App95] compares different systems of tournament on how often they determine the best player of the tournament.

Kujanssuu, Lindberg and Makinen (1999) [KLM99] apply the stable roommates problem to the Swiss system. The stable roommates problem [MW71] [Irv85] in the context of a tournament consist in establishing a ranked list of preferred opponents for each participant at each round, based on the previous results.

Olafsson [Óla90] modeled the tournament as a graph with vertices corresponding to players and edges to possible matchups according to the rules of FIDE in 1985. Then determining the pairings is done by using a maximum weight matching algorithm. This approach allows to calculate pairings and implement easily compared to the traditional algorithm. Glickman and Jensen (2005) [Mar05] use this maximum weight matching algorithm on a graph to determine the pairings, and aim to maximize the information gained from each comparison. The results show an improvement compared to random pairings and the Swiss System for 16 rounds, but in practice most tournaments only have between 5 and 11. More recently, a few works have been carried on the Swiss System in particular for team tournaments [Csa17] [Hua17]. Sauer et al. [PL23]] provide more insights on the pairing algorithm to maximize the ranking quality an the fairness of assigned colors (number of times participants have the White or Black pieces). They managed to outperform the official system with the Burstein system, and the Random2 system that they proposed. Sziklai et al. (2022) [Bal22] compare systems of tournaments with theoretical and real-world scenarios using various metrics and suggest that the Swiss System would be an interesting choice for other sports than the ones already using it.

6 Modelization of a tournament

We model a tournament by a weighted undirected graph *(G,E)*. Every vertex represent a participant, every edge a possible matchup, and weights represent the desirability of the matchup. A high value represents a game that should be played according to the defined criteria. We now denote by *players* the participants of the tournament ; they can also be teams. If there is an odd number of players, we cannot pair every participant exactly once, so one possibility is to add a vertex for a fictional player named 'bye'. The player who is paired with it doesn't play and get a free point, generally worth a victory but it is possible to give it other values (a draw, or a loss for example). The 'bye' always has 0 point. We may employ the expression "get/have a bye" for a real player who is paired with "bye".

Players cannot play against themselves, and most importantly cannot play twice the same opponent in the same tournament.

The process to run the tournament is the following :

1. Define the matrix of results, whose coefficient (i,j) is equal to the result

of player i against j (1 for a win, 0.5 for a draw, 0 for a loss or no game played yet)

2. Build the corresponding graph and determine the weight of edge (i,j) based on the matrix of results, typically taking into account stats of players i and j so far

3. Find the set of pairs such as all players are paired and the summed weights of the corresponding edges is maximized : this is equivalent to computing a maximum weight maximum cardinality matching.

4. From the previously found pairings, simulate the games

5. Update the matrix of results and repeat these steps until the planned number of rounds is done

6. Rank the players by score, then by the Tie-breaks chosen

Remark. *Let's assume that the value of "winning" against bye is 1. The number of games played in the tournament with n players (including bye) is* $\sum_{j=1}^{n} M_{ij} + M_{ji}$ *where i is any player except bye, and* $M_{ij} + M_{ji} = 1$ *if and only if players i and j have played a game together, else it's 0.*

As tie-breaks, we chose Sonneborn-Berger for round-robin tournaments. For others, we chose in this order Bucholz Cut-1 , Bucholz, - Difference of colors (quite arbitrary in our set up of simulations). The reason why we chose Bu1 is that removing the weakest player can be relevant as the first round may show high difference of levels. Therefore, some favorites might be lucky and face a much weaker opponent who will still score 2 points while another will easily crush the weakest player of the tournament with 0 point but no contribution to Bucholz. Let's now get into the details of the process.

6.1 Existence of an edge

Every edge represents a possible matchup. In the Swiss system, we do not allow two players to meet several times, so once two players are paired the edge is permanently removed from the graph. Another rule is that two players who had one color too many times compared to the other color cannot be paired together ; for example, if player i and j had both White 4 times and Black 2 times, they should not be paired to avoid a too big difference for one

of them who will again get the White pieces. In this case, we temporarily remove edge (i,j). The threshold to consider that the difference between White and Black becomes too big is arbitrary and can be changed. Here, as proposed in [Cseh], we temporarily remove the edge is the following condition is fulfilled :

$$|cd_i + cd_j| \geq 2\beta$$

where cd_i, cd_j are the difference of colors (both equal to $4 - 2 = 2$ in our example), and β is a parameter that can be changed to give more or less importance to colors, but is set to 2 by default to comply with FIDE rules. To focus really solely on ranking quality, one can remove this constraint by setting $\beta = \infty$ (in practice three times the number of rounds is clearly enough).

6.2 Weights of the edges

Olafsson [Óla90] then Sauer et al. [PL23] describe how to implement most of the previously mentioned Swiss systems at step 2 ; the weight of edge (i,j) is a tuple $(\Delta_s^{ij}, \delta_c^{ij}, \pi^{ij})$ where Δ_s^{ij} represent minus the absolute difference of score between players i and j (a victory is 1 point, a draw is 0.5 point, and a loss 0). Maximizing it means pairing people with similar number of points. $\delta_c^{ij} = -|cd_i + cd(j)|$ is the absolute difference of colors between two players ; here we prioritize people who had not the same number of times the same colors. cd_i is defined as the following as in [Cseh] : playing with the White pieces gives +1, and with the Black pieces gives -1, and cd_i is the obtained total for player i so far. Finally π^{ij} is a value that has to be chosen wisely according to the system used. It will be detailed a bit later.

Then the weights are compared lexicographically : the first criteria is to maximize Δ_s^{ij}, meaning that we want to minimize the difference of scores. Let's imagine that players a, b, c, d are in the same scoregroup : all their score differences are null. Then we study the second criteria that aims to pair people with the biggest difference in colors : ideally we want to pair people who had the more times the White pieces (cd_i is positive) with the ones who had the more often the Black pieces (cd_j negative) in order to harmonize the distribution of colors. Finally, in case of equality (for example at round 1 where no game has been played), we use the criteria that determines the system. For example, the Swiss Random pairs uses π^{ij} = a random number

in $(0, 1)$. The Burstein system uses $\pi^{ij} = |r_i - r_j|^{1.01}$ where r_i, r_j are the ranks of the players according to score, and Elo between tied scores. This prioritizes big rank differences, in particular thanks to the 1.01 exponent that makes a difference with the Dutch system (one can compare the examples given in section 3.2). It actually works with any exponent strictly greater than 1 by property of convexity. For the Dutch system, we aim to get a constant rank difference near half the size of the scoregroup : this is obtained with $\pi^{ij} = -|\frac{sg_size}{2} - |r_i - r_i||^{1.01}$, with also an exponent to penalize large distances from the desired one. The weights of the edges, including values such as the ranks, the difference of scores and colors vary across rounds, but we simplify the notations.

To implement this lexicographic ordering, one can define the weight of the pair (i, j) at a certain round as $w_{ij} = 10.000\Delta_s^{ij} + 100\delta_c^{ij} + \pi^{ij}$. Although this is done in previous works, we don't necessarily want a lexicographic order for taking into account the different values.

6.3 Maximum weight matching

With all the weights defined, we now want to determine the pairings by maximizing the total weight. Let's start with some definitions; Let $G = (V, E)$ be an undirected weighted graph.

Definition 6.1 (Matching)**.** A **matching** is a set of edges that share no common vertices.

Definition 6.2 (Maximum cardinality matching)**.** A **maximum cardinality matching** is a matching that contains the largest possible number of edges

This corresponds to making as many players as possible play at a given round.

Definition 6.3 (Perfect matching)**.** A matching is **perfect** if it covers all the vertices.

As we want all the players to play the same number of rounds, it is essential to ensure that a perfect matching exists. Pairings for a round correspond exactly to a perfect matching found on the graph at this state.

Definition 6.4 (Hamiltonian graph)**.** G is **Hamiltonian** if it contains a cycle that covers all its vertices exactly once.

Theorem 6.1 (Dirac's Theorem, 1952)**.** *Let's assume that G has $n \geq 3$ vertices. It is Hamiltonian if every vertex has degree at least $\frac{n}{2}$.*

Corollary 6.1.1. *Let's run a tournament with n players (including bye in the count) and without the color rule. If the organizer planned less than $\frac{n}{2}$ rounds, all the players will be paired at every round.*

Proof. We start the tournament with a complete graph $G = (V, E)$ as every player can face each other. V contains an even number of vertices as the case of an odd number of players is treated by adding a *bye* virtual player. Since a player is paired at most $\frac{n}{2}$ times (the number of rounds), the degree of its corresponding vertex is always greater or equal to $n - \frac{n}{2} = \frac{n}{2}$ rounds.

By Dirac's theorem, there is a Hamiltonian cycle that covers all the vertices. Let $(v_0, v_1, ..., v_{n-1}, v_0)$ be this cycle (it has length n). By taking an edge, removing the next one, taking the following until the last one, and remembering that n is even, we get the matching $\{(v_0, v_1), (v_2, v_3), ...(v_{n-2}, v_{n-1})\}$ that contains all the vertices exactly once. By pairing the players according to this perfect matching, we can conclude that at every round there exist a way to pair players who haven't played together yet. □

It is then essential to have significantly more players than rounds if we use this kind of tournament's format, which is even more true if we remove edges to satisfy color constraints.

Now that the existence of a perfect matching is ensured, we must find one to then determine the maximum weight (perfect) matching, that we will shorten into "MWM". We use Jack Edmonds' **Blossom Algorithm** [J65] for this task.

Let M a matching of G.

Definition 6.5 (Exposed vertex)**.** A vertex v is **exposed** if v isn't covered by M, meaning that no edge of the matching is incident to v.

Definition 6.6 (Augmenting path)**.** An **alternating path** is a path $(e_0, e_1, e_2, ..., e_{n-1}, e_n)$ where $e_0, e_2, e_4, ...$ are in M and $e_1, e_3, e_5, ...$ are not, or the opposite. An **augmenting path** is an *alternating path* which first and last vertex are distinct *exposed vertices*.

The goal of the algorithm is to find *augmenting paths*, and "augment" the part of the matching included in this path by the complementary. For example, if $(v_0, v_1, v_2, v_3, v_4, v_5)$ is an augmenting path with $(v_1, v_2), (v_3, v_4)$ in

M, then we can augment the matching along this path by deleting these edges from M and replacing them with $(v_0, v_1), (v_2, v3), (v_4, v_5)$. By continually augmenting paths until we can't, we reach a maximum cardinality matching as stated by Berge's theorem :

Theorem 6.2 (Berge's Theorem, 1957)**.** *A matching M is of maximum cardinality if and only if there is no M-augmenting paths in the graph.*

Algorithm 1 Blossom Algorithm for Perfect Matching (simplified)

```
Input: Graph G, initial matching M on G
Find an augmenting path P
if P is empty then
    return M
else
    Augment M along P
    Restart at the beginning
end if
Output: Maximum matching M* on G
```

Now it remains to find an augmenting path. First we pick any exposed vertex v_0 and explore the neighbours with a Breadth First Search (BFS) algorithm to determine augmenting paths. During this exploration we construct a tree T with root v_0 and different layers corresponding to different steps of the exploration. One layer is a set of vertices at equal distance from the root. When running the BFS algorithm to construct augmenting paths, we start by adding vertices attained via edges that are not in the matching M, then via edges from these vertices that are in the matching, etc. alternating edges in the matching and not. So edges in the tree from even to odd layers correspond to unmatched edges, and similarly edges from odd to even layers correspond to matched edges. Overall, when exploring a vertex v we can distinguish four cases:

1. **v is an exposed vertex** : then we get an augmenting path ; we can simply invert it and stop. We now restart from other exposed vertices.

2. **v is already matched but neither v nor its neighbour along the matched edge are in T** : we add v in the tree and his neighbour as well but in the next layer, and continue the exploration

3. **v is already in the tree and we found a cycle of even length** : we ignore and simply continue the BFS

4. **v is already in the tree and we found a cycle of odd length** : we found a blossom ; we contract it as one point and continue. Once we have an augmenting path, we expand it back and then only we can invert the path to augment it.

We keep exploring the graphs and augmenting paths until it is not possible anymore, meaning that a perfect matching was found. This is summed up in the following algorithm :

Algorithm 2 Blossom algorithm

```
1:  while There are exposed vertices left do
2:      Pick any exposed vertex v_0 and add it to the BFS queue
3:      Initialize the tree T to its root v_0
4:      while There are vertices left in the queue do
5:          Take the next vertex v in the queue
6:
7:          for all neighbours v_i of v do
8:              if v_i is matched but v_i ∉ T then                ▷ Case 2
9:                  Add v_i and its matched neighbour v'_i both in T
10:                 Add v'_i in the queue
11:             else if v_i ∈ T and cycle of even length then    ▷ Case 3
12:                 Continue
13:             else if v_i ∈ T and cycle of odd length then     ▷ Case 4
14:                 Contract the cycle into one vertex
15:                 Add it to the BFS queue
16:             else if v_i is exposed then                      ▷ Case 1
17:                 Expand the contracted blossoms
18:                 Invert the resulting augmenting path
19:             end if
20:         end for
21:     end while
22: end while
23: return Maximum matching
```

This algorithm has complexity $O(|E|.|V|^2)$. To compute the maximum weight matching, we use the library **networkx** which documentation con-

tains reference on how to use the Blossom algorithm in the case of weighted matchings [24a].

7 Set up for simulations and tests

The number of rounds of a tournament is fixed before the start, as well as the list of players. Each player will be represented initially by their name, Elo (official/displayed estimation of playing strength), strength (the real level of the player), and attributes describing their performance : number of points, values of Tie-breaks, but also the difference of colors (=number of games the player has played the White pieces - number of games played with the Black) pieces

7.1 Elo rating vs playing strength

In real life, players have an **official rating (Elo)**. Some players don't have an official rating (they are called *unrated* players). Here we consider that only players with low level don't have a rating, which is a logical assumption but it is more complex in practice.

The actual **playing strength** is of course correlated with the official rating, but this is not an exact estimation. It depends mostly on the frequency the players participate in rated tournament, but as we consider the strength of a player during the time of the tournament, it also represents their current form and focus during the short period of the event. The playing strength is unknown in practice, but is a relevant information to use in the context of simulations. This is why using real-life tournaments' outcomes is not relevant to evaluate a system.

The strengths of the players are assumed to be normally distributed around 1600 and standard deviation 400.

The Elo of the players are computed from their strength as in Sauer et al. [PL23], via a normal distribution centered around their strength and of standard deviation $\frac{3000-\text{strength}}{20}$. This dependence on the *strength* translated the fact that players of high level generally play more often which is why their Elo should be reliable and closer to their playing strength. We may try different values to test Elos that are further from the strengths, or even without any rating at all (Elo always equal to 1000 for example) so our results can be directly applied to sports where there is no such rating system. We

could also try different distributions for playing strengths but we will stick with this in our project.

To simulate the matches, we follow the theoretical definition of the Elo system, which is the one we use for strengths. Let D be the absolute difference of strength between the two players. The expected result $p(D)$ of the strongest player (1 meaning a win, 0 a loss) is given by :

$$p(D) = (1 + 10^{\frac{-D}{400}})^{-1}$$

As this is the average score of the strongest player but there is also a probability of a draw, we arbitrarily define the probability of a draw as $p_{\text{draw}} := 1 - p(D)$, so :

- The probability that the strongest player wins is $p(D) - \frac{p_{\text{draw}}}{2} = \frac{3p(D)}{2} - 0.5$, which is non-negative as $p(D) \geq 0.5$ and smaller or equal to 1 because $p(D) \leq 1$
- The probability of draw is $p_{\text{draw}} := 1 - p(D)$
- The probability that the weaker player wins is $1 - p(D) - \frac{p_{\text{draw}}}{2} = 0.5 - \frac{p(D)}{2}$ which is non-negative because $p(D) \leq 1$ and smaller or equal to 1 because $p(D) \geq 0$.

Additionally in these simulations, we set to 1000 any Elo below to represent unrated beginners, so a player with a 500 strength will probably have 1000 Elo.

7.2 Number of players and rounds

Unless stated otherwise, we take the same parameters as in [Cseh] and some other works : $n = 32$ players, and $r = 7$ rounds. This allows to reasonably represent real-life tournaments that generally have between 7 and 9 rounds, with a number of players between $2r$ (Dirac's theorem) and 2^r (to allow finding the best player in time), and allow relatively short computational time as the Blossom algorithm has quadratic complexity with respect to the number of players and linear in the number of rounds.

We may vary the pool of players for some tests of comparison between systems by generating between 30 and 80 participants per tournament. The different systems of a same comparison will be tested on the same pools of players. The number of rounds will stay equal to 7 unless stated otherwise.

7.3 Assigning colors

As mentioned in section 6.1, we set the condition

$$|cd_i + cd_j| < 2\beta$$

for the matchup to be an option, where cd_i, cd_j are the difference of colors (games played as white - games played as black) of players i and j, and β is a parameter set to 2 by default. It is also possible to remove this condition ($\beta = \infty$) to always let all the possibilities and pairings, and rely on the fact that players who had one color more than the other should automatically compensate their color difference with other players : when two players are paired, we give the White pieces to the player who had the most times the Black pieces so far. In our tests, although we will keep the condition with $\beta = 2$ for the edge to exist, we will remove δ_c from the calculations of the weights of the edges. This is to ensure fair comparison between the systems, as our future NPS system will not take colors into account in the scope of this project. Therefore, we will simply assign colors once the pairs are decided as we just described.

7.4 Metrics to evaluate a system of tournament

Let's exclude the *bye* player (who is only used for pairing purpose) and extract from the final rankings the list of the players' strengths, keeping the same order. This is a vector $v_r \in \mathbb{R}^n$ where n is the number of players. Let v_t be the theoretical ranking of strength, which is the list we try to get as close to as possible. It is clearly impossible to consistently get this exact perfect ranking with an hypothetical perfect system, because the results of the individual games are random.

Many metrics can be used to evaluate a system ; International Arbiter Mario Held [Hel20] used Average Rating Opposition (ARO) of players within each scoregroup to check whether they had faced a more or less equal opposition. Hua [Hua17], Sauer et al. [PL23] used correlation measures such as the Kendall τ defined as follows. Let $x = (x_1, x_2, ..., x_n), y = (y_1, y_2, ..., y_n)$ where each vector has n distinct values. In our context, values of y are just values of x but reordered differently. A pair of points (x_i, y_i) and (x_j, y_j) with $i < j$ is *concordant* if both $x_i < x_j$ and $y_i < y_j$, or both $x_i > x_j$ and $y_i > y_j$. This means each coordinate of one of the points are greater than both coordinate

of the other point. Else the pair is called *discordant*. By browsing all pairs of points, we can compute the **Kendall Tau** :

$$\tau(x, y) := \frac{\#\text{ Concordant pairs} - \#\text{ Discordant pairs}}{\#\text{Pairs}}$$

The values lie between -1 and 1, and we aim to get high positive values, meaning great correlation between our obtain ranking v_r and the target v_t.

To treat the case of ties between values, we will use a slightly modified version, called **Tau-b** that we won't elaborate here but works similarly.

Simple metrics that describes how close two vector can be used ; for example the **Euclidean distance** also takes into account how big an inaccuracy in the rankings is. An inversion of a 1500 rated player with a 2000 will be more costly than a 1500 with a 1600. In the Kendall τ, this is not the case if there is no participant rated between 1600 and 2000. Although such cases may show a disadvantage for the Kendall τ, in practice there should be players rated between these values. We will use both to evaluate our rankings. Instead of the Euclidean distance, it is of course possible to use the Manhattan distance that gives "linear" penalties instead of punishing more severely larger differences of rating.

Our second goal was to optimize the ranking of the top 5 players ; we distinguish two problems.

- **top-k** : Rank the k best players accurately, with more emphasis on the 1st, then 2nd etc. This is particularly suited to tournaments giving qualifying spots or money prizes to the first k players. To evaluate our systems with respect to this goal, we propose two metrics based on the ones mentioned for general ranking. The first one is a kind of weighted Manhattan distance which we will call **top-k Ranking Weighted Evaluation (RWE)** defined as :

  $$\text{RWE}_k(x, y) = \sum_{i=1}^{k} (k - i + 1)|x_i - y_i|$$

  for x and y two vectors representing rankings (v_t and v_r but only the first k values are necessary). This gives linearly decreasing weight to rating difference from 1st place to k-th.

- **select-k** : Find the k best players accurately, but without insisting on ranking them correctly ; for example if $k = 3$ and the players from

strongest to weaker are $1, 2, 3, 4, 5$, having $1-2-3-4-5$ represent the same success as $3-1-2-4-5$ because the top 3 contains all the 3 best players. However $1-2-4-3-5$ must be better than $1-3-4-2-5$ if 2 has a strictly greater strength than 3. Hence we give greater penalty if very weak players sneak into top k, or if the strongest players miss. The penalty will be small if players that should not be in top k are in top k but of almost the same level as the last players who should be in. To evaluate a system we can simply sum all the strengths of the k first players' that should be in, that are actually in, and take the difference to get the "**missing strength**".

We can also adapt the Kendall Tau as a *Weighted Kendall* τ [24e] giving more importance to the first places, and little to no importance to the rest of the ranking. It may be useful to assume that in practice, as looking for the best player often results in a decent overall ranking, looking for the k best players should also yield a good ranking. Conversely, finding a great system for better overall ranking may help with these problems.

For goal 3, computing the weighted Kendall Tau on the ranking of a subcategory of players looks relevant (we are generally only interested on the 1-3 first places) but for simplicity we will use the **missing strength** with $k = 3$.

8 Evaluation of the base systems

Let's first evaluate the different systems that we already have. The worst system that still makes sense in the **Random** system where pairings are chosen totally randomly. It will be our lower bound in terms of quality (we keep the term "lower bound" even for metrics that must be minimized) in the following comparisons. As the matchups often result in the stronger player winning, we expect a clearly positive correlation measure between the obtained ranking and the theoretical one. An upper bound would be the **Round robin** system where every player plays against every other one. The number of rounds is special here as it is not 7 but $n-1$ where n is the number of players, generally 32. We don't expect that the system with 7 rounds can get any close to this value, but it gives an idea of what range of values is impossible to reach.

We also include :

- The **Monrad** system
- The **Dutch** system, the official and most popular one
- The **Burstein**, the state-of-the-art, also official
- The **Swissrandom**, different from the Random as in the Swissrandom we pair randomly within scoregroups, not in general
- The **Swissrandom2** defined in Sauer et al. [PL23]

We often fixed the seed at 77 for reproducibility (c.f. code). Complete results can also be found in the appendix.

8.1 General ranking

Average values over 10000 tournaments with 32 players :
The worst tau-b should be 0.7209 and the ideal is 0.8898
Swiss Monrad : 0.6384
Swiss Dutch : 0.7778
Swiss Burstein : 0.8019
Swiss Random : 0.7642
Swiss Random2 : 0.7669

The worst Euclidean distance should be 1021.0046 and the ideal is 381.2911
Swiss Monrad : 1172.5766
Swiss Dutch : 783.7927
Swiss Burstein : 714.7064
Swiss Random : 815.6196
Swiss Random2 : 808.5458

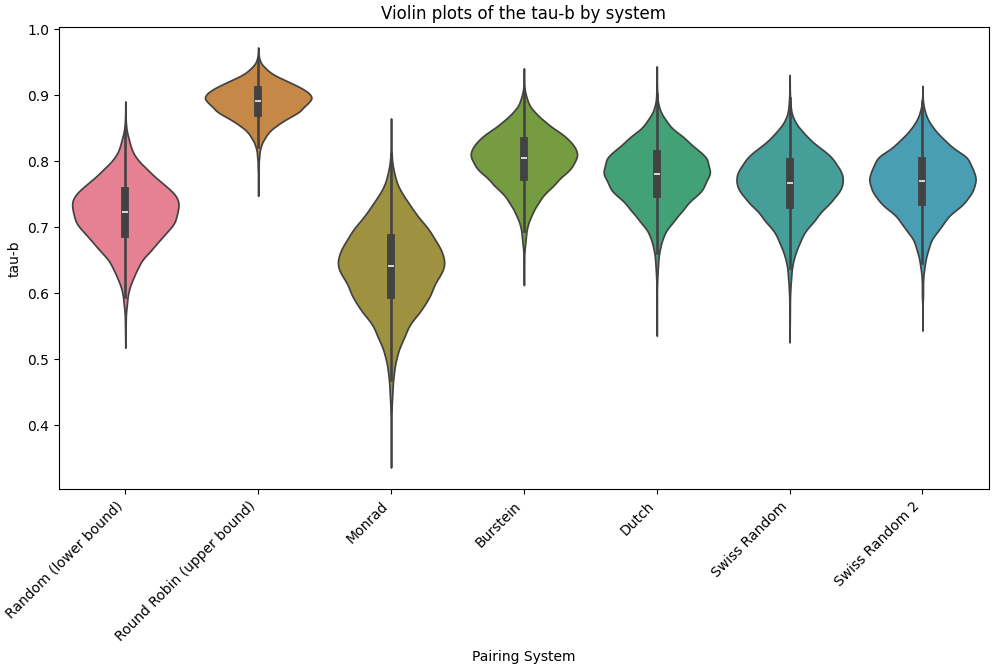


Figure 3: General ranking quality measured by Kendall τ_b correlation, values close to 1 means better ranking quality. 10000 simulations with 32 players

Figure 3 shows the density of correlations between the obtained ranking and the reference ranking (players ranked by strength) for each system. Surprisingly, the **Monrad** system performs even worse than the **Random**. The **Round Robin** is of course by far the best with values around 90%. Then the **Burstein** is the only one above 80%. The **Dutch**, looks like a slightly improved version of the two **Swissrandom**. **Swissrandom2** may be the best of them, but the difference is thin. This would however tend to confirm results from Sauer et al. [PL23]. The whole hierarchy is identical

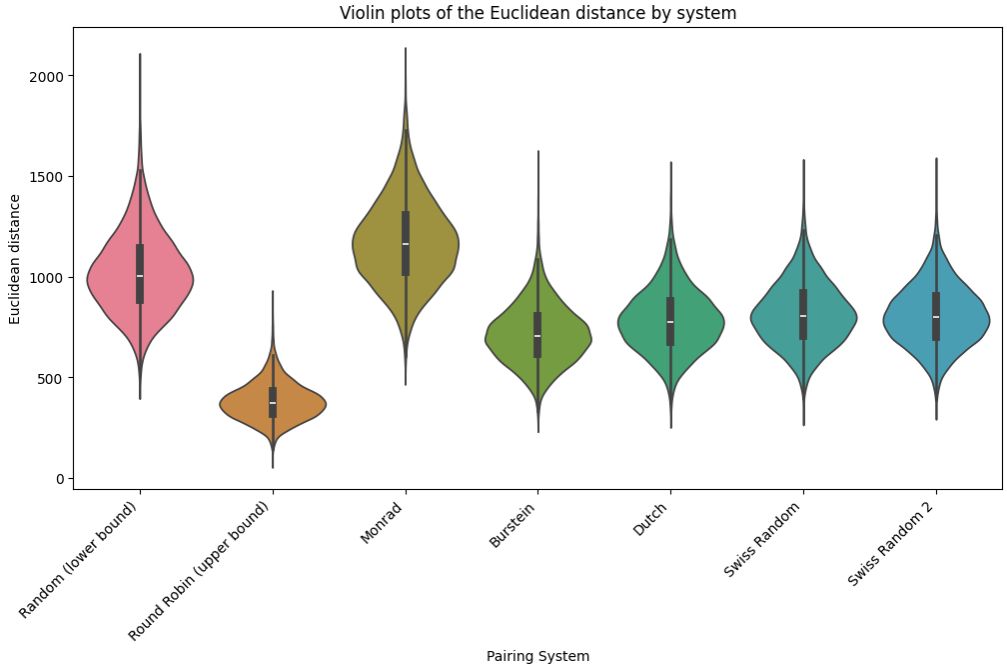


Figure 4: General ranking quality measured by Euclidean distance, low values means better ranking quality. 10000 simulations with 32 players

by interpreting the results obtained from computing the Euclidean distance (Figure 6).

8.2 Top k

We now evaluate how well the systems rank the best 5 players. We again report the average values after 10000 simulations :

> The worst top 5 RWE should be 2119.3695 and the ideal is 607.1923
> Swiss Monrad : 1428.7546
> Swiss Dutch : 1146.3817
> Swiss Burstein : 1123.4811
> Swiss Random : 1169.0321
> Swiss Random2 : 1147.1425

The worst missing strength should be 322.2074 and the ideal is 53.8895
Swiss Monrad : 373.3742
Swiss Dutch : 216.1629
Swiss Burstein : 164.5964
Swiss Random : 210.7405
Swiss Random2 : 202.4643

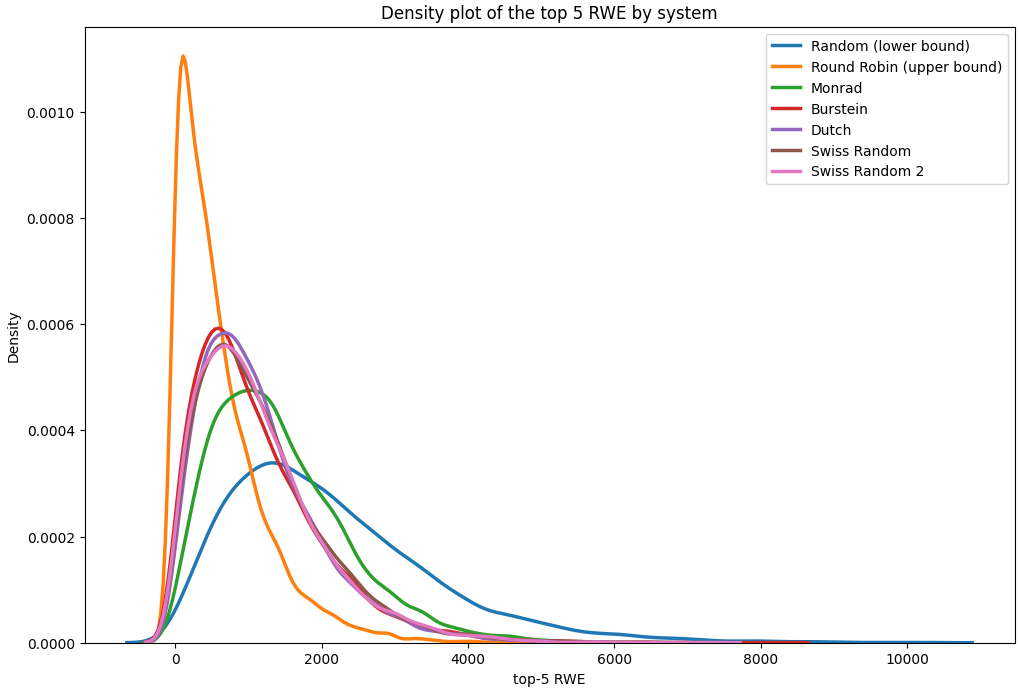


Figure 5: Top 5 ranking quality measured by top-5 RWE, high density near 0 means better ranking quality. 10000 simulations with 32 players

The results are very similar to the general ranking as the hierarchy is almost the same with the **Burstein** confirming its lead. The **Monrad** system seems to prove that it is not worse than Random pairings when it comes to ranking the best players. It still ranks behind the other base Swiss Systems. The **Swiss Random 2** this time doesn't look worse than the **Dutch** as it barely behind according to *top-5 RWE* and even better at minimizing the select-5 missing strength.

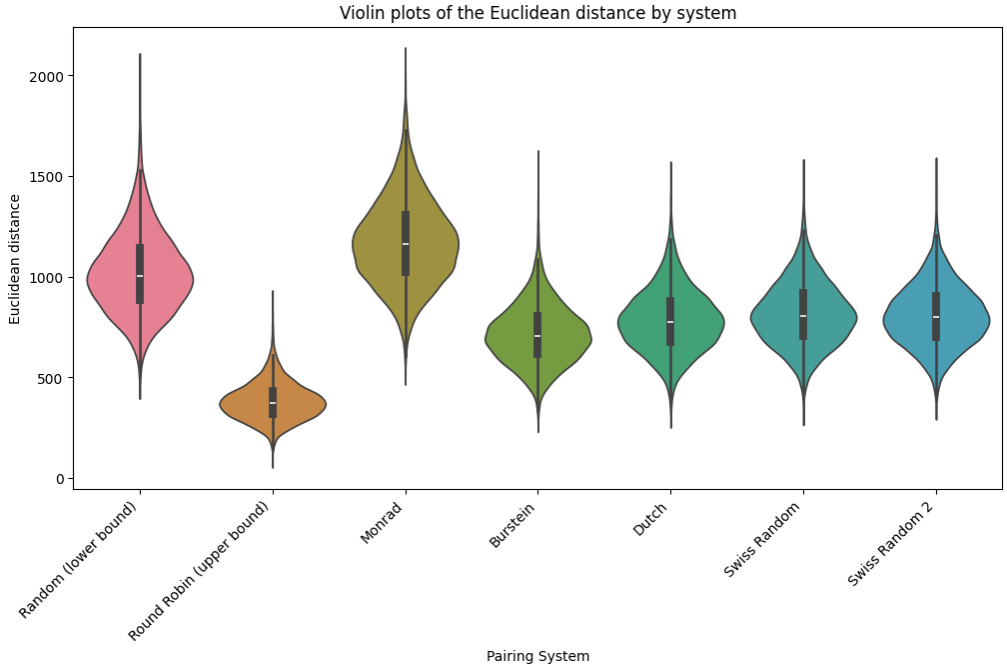


Figure 6: select-5 ranking quality measured by missing strength, low values means better ranking quality. 10000 simulations with 32 players

8.3 Subcategories

We introduced a subcategory representing in average 20% of players and studied how well the best 3 would be found.

> The worst missing strength should be 74.5251 and the ideal is 11.9587
> Swiss Monrad : 138.7758
> Swiss Dutch : 48.9096
> Swiss Burstein : 40.6132
> Swiss Random : 58.7714
> Swiss Random2 : 55.1901

First of all it appears from all these tests that the round robin is extremely good compared to the 7-rounds tournaments. While goal 2 is related to the system's performance at the top of the ranking, goal 3 is more related to producing a decent ranking at the middle. The **Monrad** again does worse

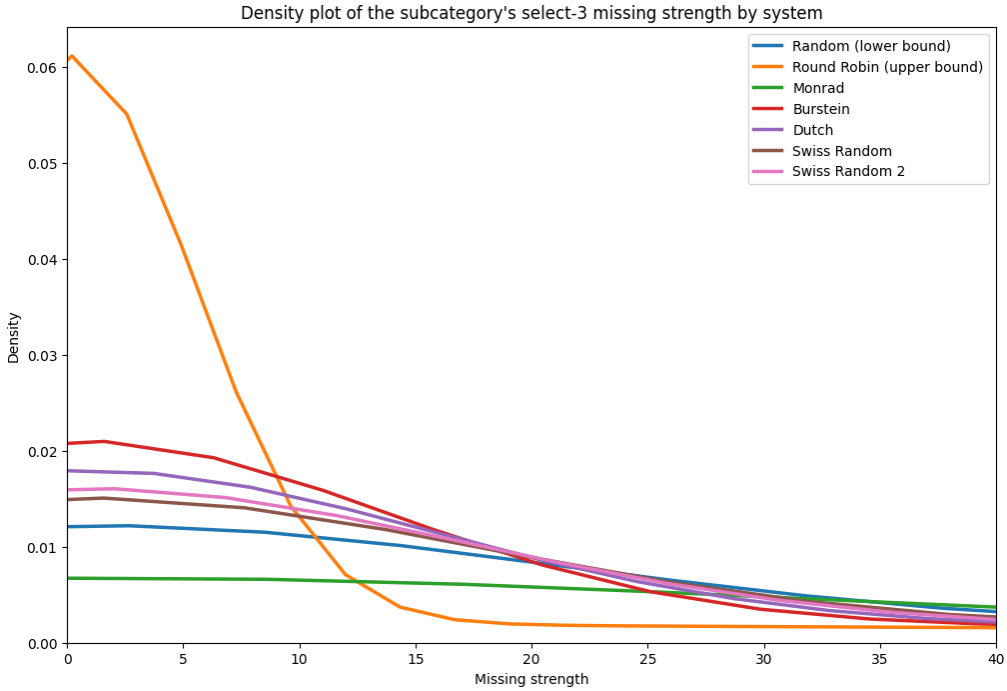


Figure 7: Subcategory select-3 ranking quality measured by missing strength, high density close to 0 means better ranking quality. 10000 simulations with 32 players

than the lower bound, while the **Dutch** is the best apart from the **Burstein** (and obviously the upper bound). None of these systems are really suited for this task and the explanation for their performances looks more related to overall quality.

8.4 Ideal number of rounds

As we mentioned earlier, given a tournament with $n \geq 4$ players, the number of rounds r must satisfy the relation $log_2(n) \leq r \leq \frac{n}{2}$. For 32 players, this means we must choose a number of rounds between 5 and 16. Generally, organizers put limitations on the number of players for logistic reasons and determine a reasonable number of rounds in advance for the expected number of participants. It is interesting to check how well the different systems perform across the possible number of rounds, in order to

determine the ideal number or rounds or ideal system for a given number of rounds.

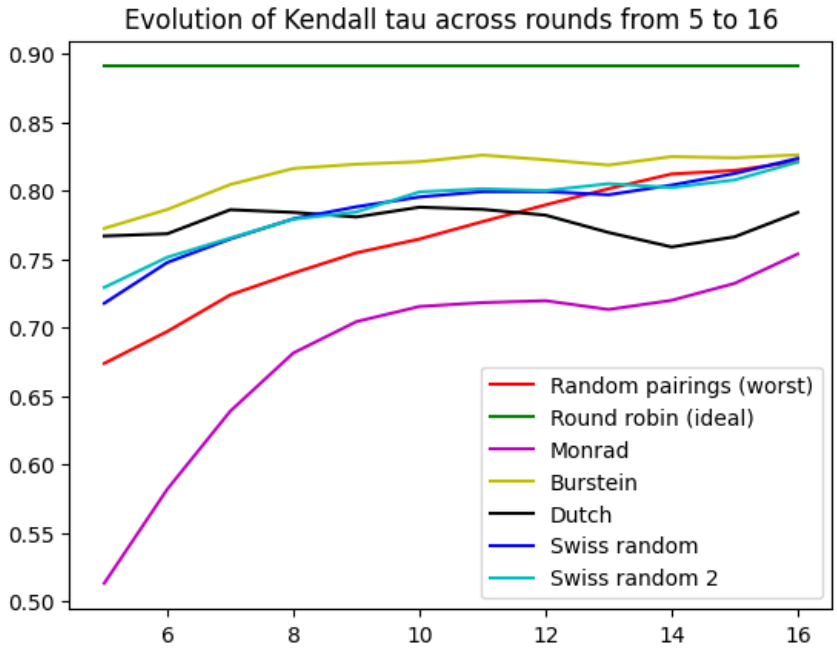


Figure 8: Average Kendall τ_b for different systems with 5 to 16 rounds, 200 simulations with 32 players, seed 77

The most informative result from Figure 8 is the fact that the official system (Dutch) inevitably falls behind the "naive" Random system as the rounds progress ! It appears clearly that there are more conditions on the number of rounds to ensure the good performance of the Dutch system. The point of the Dutch and Burstein system, in opposite to the Swiss Random and Random, is that we reach very quickly a good performance (between $log_2(n)$ and $log_2(n) + 3$) Fortunately, an arbiter would likely never put more than 9 rounds in a 32-players tournament, so with a modest and usual number of rounds, the Dutch system performs decently. The Monrad system shows very poor results all along. The Swiss Random family seems to continuously improve until reaching the best values. The Burstein system performs consistently the best and is only tied after 16 rounds.

8.5 Oscillating behaviour of players across rounds

Many players in Dutch systems experience a "yo-yo" phenomenon ; when they win a game, they face a stronger player, to whom they lose, then are paired against a weaker player and win. They repeat the process until reaching the end of the tournament ; their final placement, above or below their theoretical rank, seems to strongly depend on their last result (winning at the end would likely result in a "too high" rank). Overall it looks like players oscillate a lot in Swiss systems, and we can try to quantify this phenomenon. Let's first visualize positions' changes of some players :

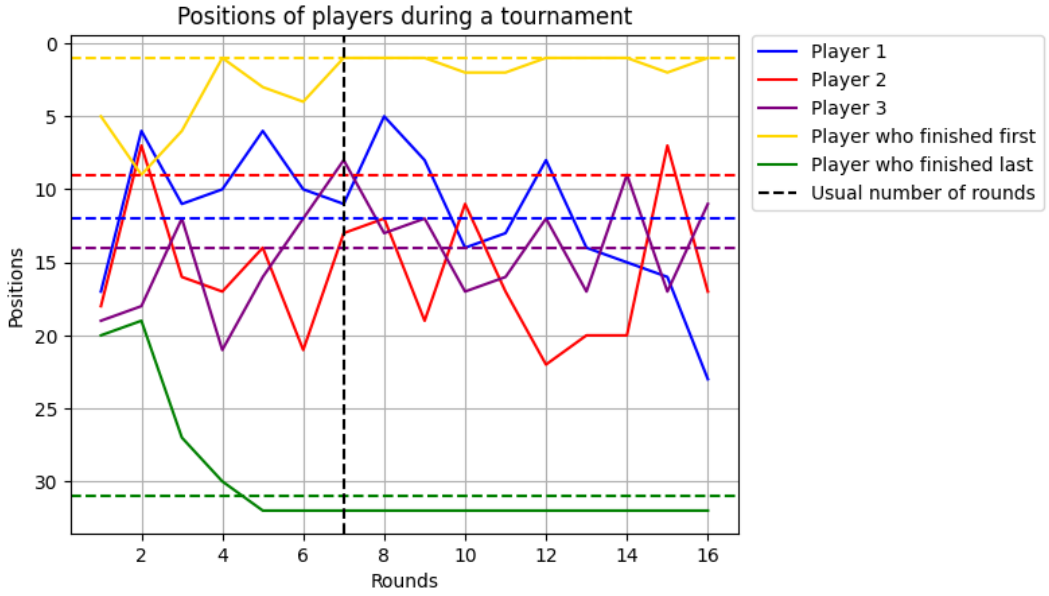


Figure 9: Evolution of the positions of 5 players, including the winner, the one who finished 32th/32, and 3 randomly picked, during 16 rounds. In dashed horizontal colored lines, the expeted/theoretical rank of the corresopnding player.

We simulated 16 rounds to evaluate what would happen after the 7 rounds that we set by default as a reasonable number. It looks like the players were far from stabilizing themselves, apart from extreme cases who may be players who are far too strong (or the contrary!) to get inconsistent results. The selected players seem to move around their theoretical rank. An appropriate

attempt to solve this problem could be to change the number of points given at each round. We can make the parallel with alternating series.

In the Swiss systems so far, players could win to a weaker opponent, then lose because paired against stronger, and alternate. The sum of their points minus half of the maximum possible number of points would be a sum of +0.5 and -0.5. That sum would never converge. If the number of points allocated to each round decreased, by the alternating series test, the final number of points would converge.

This reasoning is of course very inaccurate, but it is meant to show the logic behind the first system that we introduce.

9 Introducing the Pheng System

To address the issues related to the oscillations, an idea is to reduce the (relative) number of points allocated across the rounds. For example, one could set the initial number of points to 3 for the first rounds, then decrease it to 2 and finally 1. Additionally to the arguments from the previous section, this measure would compensate for the importance of every round : in the regular swiss system(s), losing the first game is not dramatic for a player who aims for the first half of the ranking as it will result in having easier pairings for a few rounds. The last round is however crucial, as winning will not be "punished" with hard pairings right after, and losing will not give easier opponents either. Rewarding 3 pts at round 1 would force the participant to not want to get into a position where winning several games in a row is necessary, and rewarding only 1 pt at the last round would reduce the consequences of the end of the tournament.

The player who is familiar with variations of the Dutch system may ask how this is fundamentally different from accelerated pairings. In accelerated pairings, fictional points are given at the beginning to players with higher Elos. These points are then progressively removed across the rounds. It reduces the gap between paired players and allows the find the best player in less games than the regular systems. However this is not trying to establish a better overall ranking and players with low Elos will easily win real points to get constantly close to high rated players who have more fictional points. As the value of a round is still at its highest at the end of the tournament, it is logical that we observe the same "singularities" at the border of scoregroups in accelerated pairings as in the Swiss Dutch.

The goal is now to determine the best sequence of points $(x_i)_{i=1}^{7}$ of length 7 (the number of rounds) where x_i is the number of points given at round i. In other words, the winner will gain x_i points, the loser 0, and in case of a draw each player will get $\frac{x_i}{2}$. The value of a bye is set to the value of a draw ; a win would be a too strong choice as the player receiving the bye is generally at the bottom of the ranking (especially if round 1 rewards a lot of points). The default sequence, $(1, 1, 1, 1, 1, 1, 1)$, is equivalent to $(2, 2, 2, 2, 2, 2, 2)$ and $(3, 3, 3, 3, 3, 3, 3)$ as it would give the same algorithm but with points multiplied by a constant. As we want to use this system in practice, we will only keep integer values. The numbers should also be coprime (meaning that we would choose $(1, 1, 1, 1, 1, 1, 1)$ over $(2, 2, 2, 2, 2, 2, 2)$ or $(3, 3, 3, 3, 3, 3, 3)$).

Although it is possible to try large values for points (and end up with lists like $[27, 14, 17, ...]$), we choose to keep these numbers between 1 and 3 (included) as it is unlikely that players would enjoy complicated sequence of points. This reduces a lot the possibilities, but still includes interesting options. By testing every possibility, we find that $(3, 2, 1, 1, 1, 1, 1)$ is the best if we pair like a Dutch or Burstein system in case of equality of points. Any sequence with high values for early rounds and 1 for the last ones seems to perform very well. After 1000 thousands simulations with 30 to 80 players by tournament, we get the following results (Kendall tau-b) :

- The worst performance should be 0.7086 and the ideal is 0.9132 (based on the random pairings and round-robin format)
- Swiss Dutch : 0.7894
- Swiss Burstein : 0.8069
- Swiss Pheng (Dutch): 0.8254
- Swiss Pheng (Burstein) : 0.8276

As the Burstein's way to compute the weights seemed to be the best, it seems logical that this format is also best applied to the Pheng system. We will now implicitly use it without precising every time.

9.1 General ranking

We first check the effect of a decreasing sequence of points on the oscillations described in section 8.5, see Figure 10.

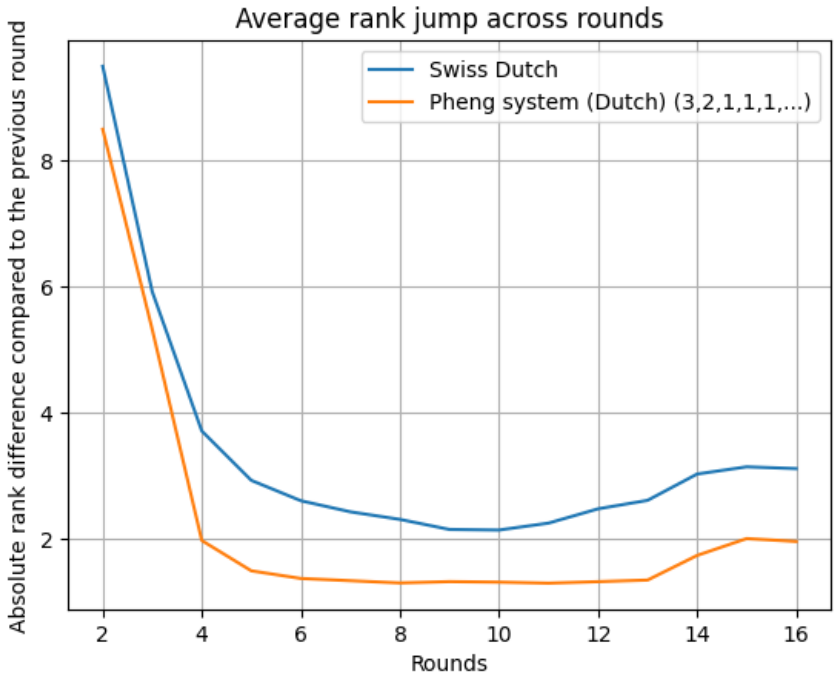


Figure 10: Average size of players' jumps in the ranking compared to their ranking at the previous round

We see that we have managed to effectively reduce the magnitude of positions' change, resulting in a more stable ranking. The chosen sequence of points seems effective until 11 rounds.

Now let's check the ranking quality :

The worst tau-b should be 0.7209 and the ideal is 0.8898
Swiss Dutch : 0.7778
Swiss Burstein : 0.8019
Swiss Pheng : 0.8360
The worst Euclidean distance should be 1021.0046 and the ideal is 381.2911
Swiss Dutch : 783.7927
Swiss Burstein : 714.7064
Swiss Pheng : 593.4126

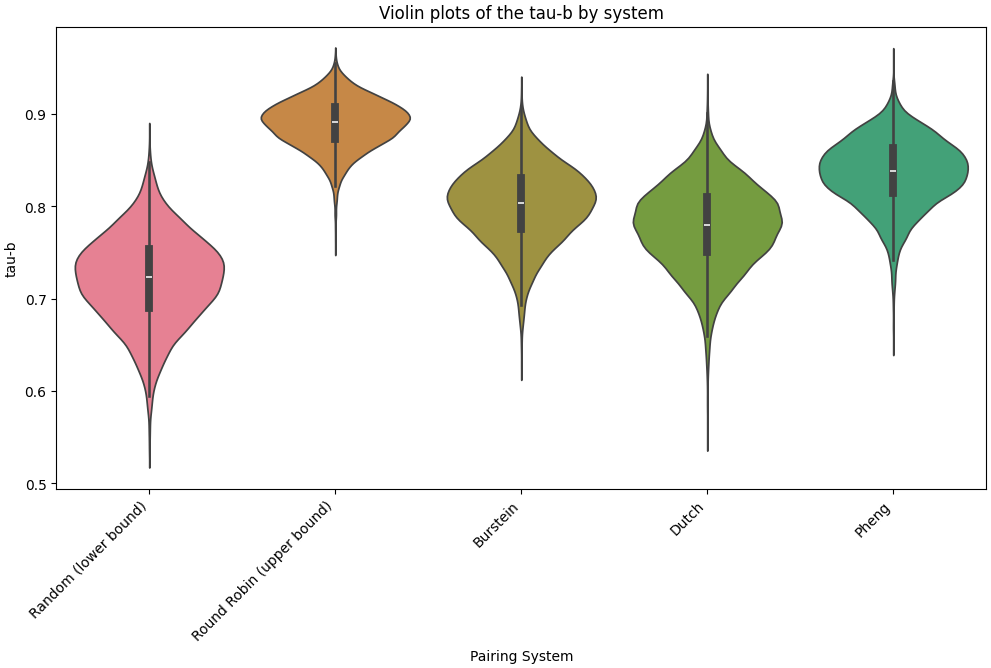


Figure 11: General ranking quality measured by Kendall τ_b correlation, values close to 1 means better ranking quality. 10000 simulations with 32 players

We observe tremendous improvement with this system, as it significantly beats the state-of-the-art 7-rounds system using the same number of rounds. It is important to note that the optimal sequence of points must be adapted to the length of the tournament, and probably other factors such as the dispersion of strengths, ratings, number of participants etc. It seems however that a lot of decreasing sequences performs well, so as long as 7 rounds is appropriate for the number of players, the given sequence could be efficient enough.

9.2 Top k and Subcategories

Although this system do not directly address these questions, the general improvement of the ranking quality may obviously impact the secondary goals. Let's start with ranking the best 5 players.

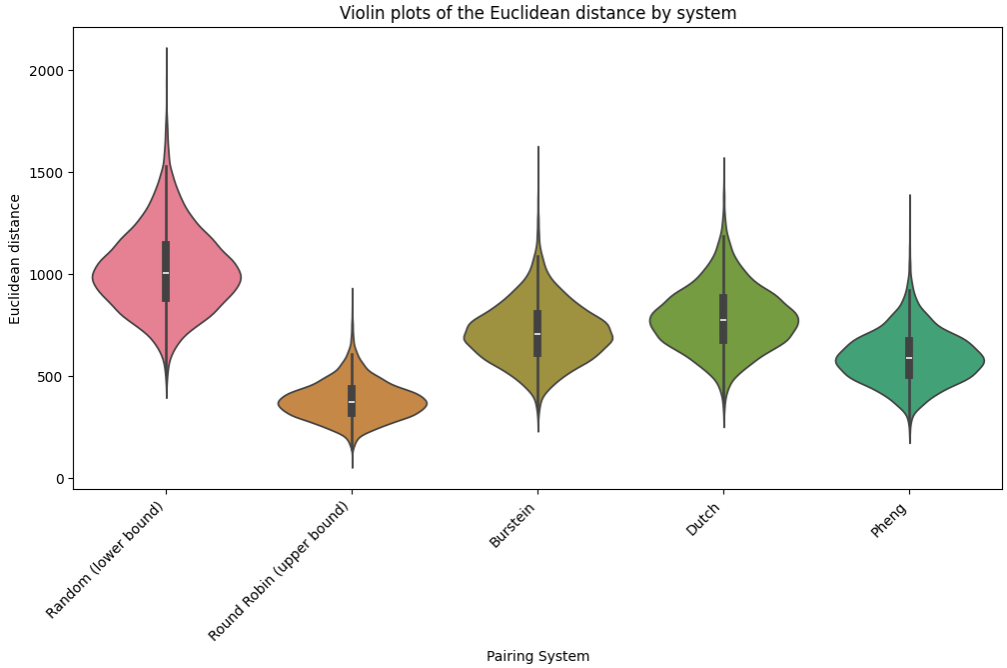


Figure 12: General ranking quality measured by Euclidean distance, values close to 0 means better ranking quality. 10000 simulations with 32 players

The worst top 5 RWE should be 2119.3695 and the ideal is 607.1923
Swiss Dutch : 1146.3817
Swiss Burstein : 1123.4811
Swiss Pheng : 985.9918
The worst missing strength should be 322.2074
and the ideal is 53.8895
Swiss Dutch : 216.1629
Swiss Burstein : 164.5964
Swiss Pheng : 114.1693

Now let's check the performance on ranking a subcategory.

The worst missing strength should be 74.5251 and the ideal is 11.9587

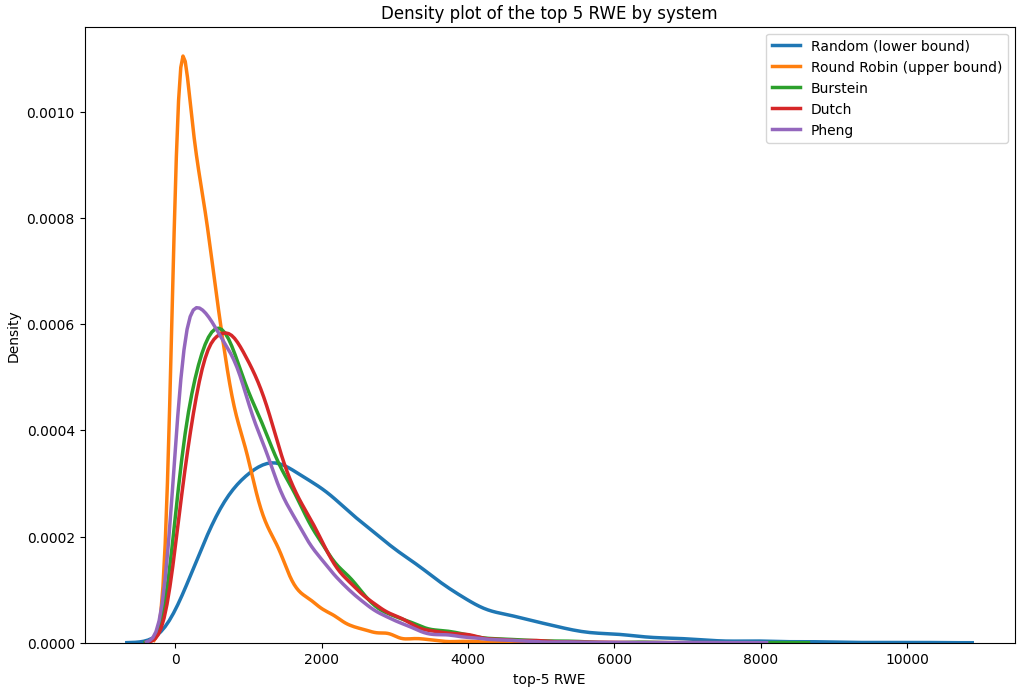


Figure 13: Top-5 ranking quality measured by top-5 RWE, high density near 0 means better ranking quality. 10000 simulations with 32 players

Swiss Dutch : 48.9096
Swiss Burstein : 40.6132
Swiss Pheng : 27.8085

The qualities of this new system impact as expected all the secondary goals that we fixed ourselves ; it would be interesting to keep improving this system by finding new methods.

9.3 Effect of seeding

It is important to take into account initial seeding when pairing, else strong players may face each other in a 3-points match, then again in another 2-points match. This random effect may increase the probability of a bad quality ranking. However a lot of sports don't have systems like Elo's, or organizers could prefer not to take Elo into account to make pairings for

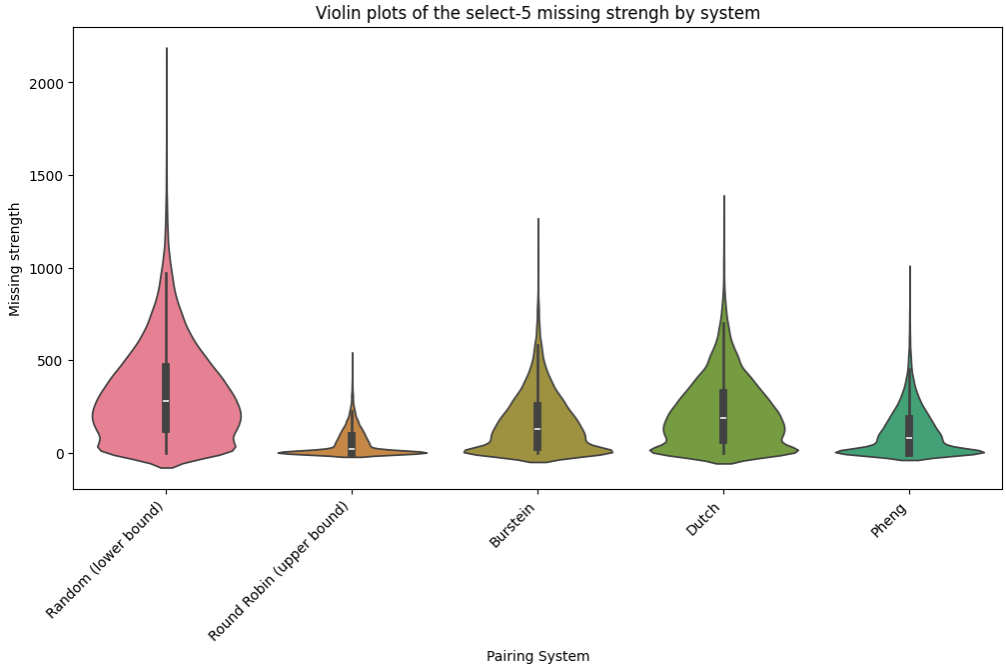


Figure 14: Select-5 ranking quality measured by missing strength, values close to 0 means better ranking quality. 10000 simulations with 32 players

example to avoid players with higher ratings to benefit from any advantage. This means that round 1 has to be random since we do not have any information of the players. After round 1 we do not have more information than the number of points, but we have more after round 2 : this is given by the tie-breaker, as the previous opponents of a player have played at least another game with measurable success. We can then use the Tie-break value instead of Elo. We compare now the Pheng system as defined previously ("With seeding (Elo)") with the same one but without seeding and by adapting the sequence of numbers ("No seeding (Random)"). Then we introduce the Bucholz cut-1 criteria to replace the Elo one, still in Burstein system (within a scoregroup, pair the highest Bu1 with the lowest). This is an interesting choice as the highest Bu1 of the scoregroup will easily access to the upper scoregroup by facing the supposedly weakest player of the scoregroup, and vice-versa. This idea corresponds to the third column in table 1. We reported the average values of the tau-b and Euclidean distance for the general rankings, top-5 RWE and missing strength (select-5) for the best 5 players

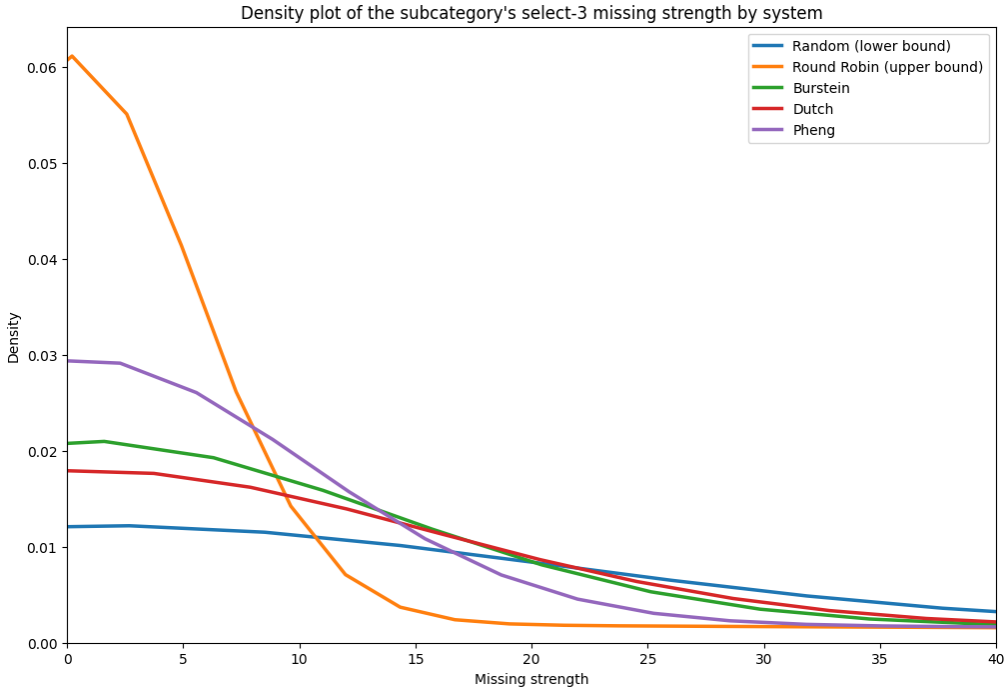


Figure 15: Subcategory ranking quality measured by Euclidean distance, values close to 0 means better ranking quality. 10000 simulations with 32 players

(goal 2), and the missing strength (select-3) for the best 3 players of the subcategory, which was drawn randomly by taking 5 to 10 players. The optimal sequences of points is defined as the one that maximizes the Kendall τ_b.

From the *Optimal sequence of points*, it looks like the system is - as expected - more careful when there is no seeding : either by never giving 3 full points for a game, either by spreading high values until the end of the tournament.

The effect of seeding is very clear on this pairing system ; since it is harder to predict when players with close levels will be paired, the importance of rounds is more complex to understand. The Pheng system does not exactly bring a solution to tournaments without seeding, but it is still slightly more efficient in this case as shows Figure 16. The average values obtained for the comparison are the following :

	With seeding (Elo)	No seeding (Random)	No seeding (Bu1)
Optimal points	3-2-1-1-1-1-1	2-2-1-1-1-1-1	3-3-2-2-2-2-1
tau-b (%)	83.68 (36.99)	77.35 (48.76)	77.24 (48.06)
Euclidean distance	592.3 (128)	786.3 (153)	788.6 (153)
top-5 RWE	992 (844)	1160 (868)	1183 (892)
select-5	115.4 (124)	200.6 (179)	201.2 (180)
subcat select-3	32.12 (67.4)	65.65 (111)	66.54 (113)

Table 1: Optimal sequences of points, and results of tests for all three variations of the Pheng system. The first number corresponds to the average value over 10000 simulations of 32-players and 7-rounds tournaments. The number in parenthesis is the standard deviation.

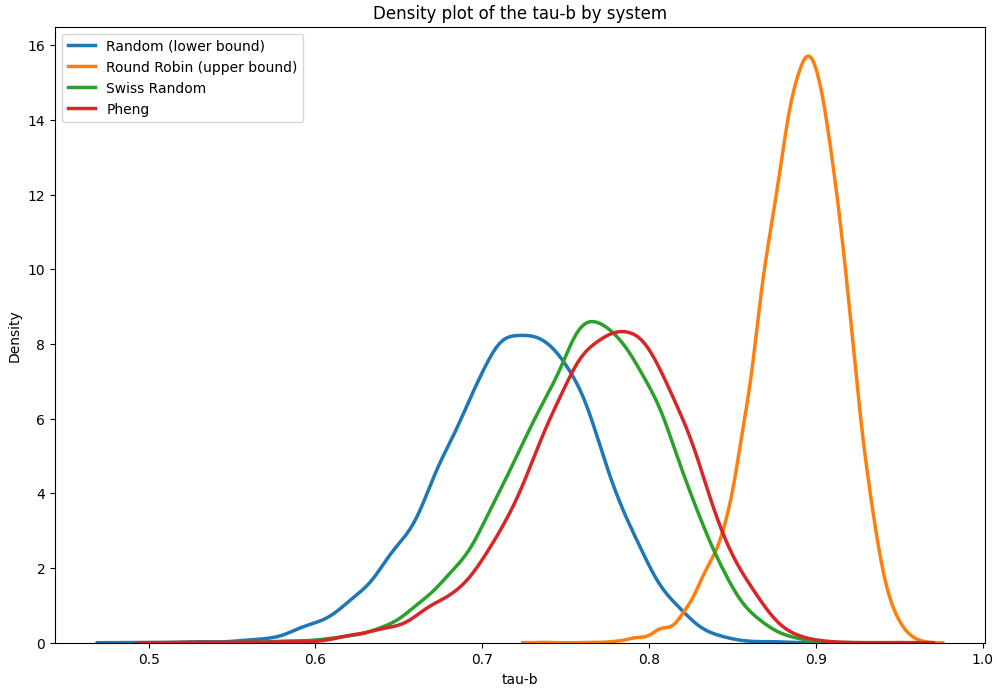


Figure 16: General ranking quality measured by tau_b on systems without seeding (every base Swiss System is equivalent to Swiss Random if there is no rating involved), high density near 0 means better ranking quality. 10000 simulations with 32 players

The worst tau-b should be 0.7202 and the ideal is 0.8898
Swiss Random : 0.7634

Swiss Pheng : 0.7735
The worst Euclidean distance should be 1027.7146 and the ideal is 381.5615
Swiss Random : 821.1070
Swiss Pheng : 788.8788

The introduction of Bu1 did not improve the choice of opponents within scoregroups. Hence in the usual case with seeding we use the first version, but without seeding we would use the random version of the Pheng system instead of using another one involving Bu1 ranks.

10 Generalizing using an AI approach : the Neural Pairing System

Although the Pheng system present great results, it would be more easily accepted that a new system keeps rewarding a constant number of points. So far, Swiss systems pair in priority players with the same number of points (as far as possible), and use other factors in case of equality. As a reminder, the formula for the weight of an edge (i, j) at round r, once the term for color balance ignored in our context is the following :

$$w(i, j, r) = 10000 * \Delta_s(i, j, r) + \pi(i, j, r)$$

where $\Delta_s(i, j, r) = -|\text{score of i at round r} - \text{score of j at round r}|$ and $\pi(i, j, r)$ is a function of Elo ranks of i and j within the scoregroup if i and j are in the same scoregroup, depending no the system used, and 0 is i and j are not in the same scoregroup.

10000 and 1 seem arbitrary coefficients and could be optimized : it would allow the system to not systematically pair people with the same number of points. It would be also interesting in practice to replace the Elo dependence : firstly, to try using different criterias like ranking scoregroups by tie breaker instead of Elo. Secondly to avoid possible advantageous pairings for players with high Elos, giving them more chance to finish at the top but for wrong reasons. Thirdly, to make this system relevant for sports that doesn't use the Elo system. The goal will be therefore to build a formula for the weight of an edge by ;

- Varying the importance of each term instead of lexicographical comparisons
- Varying the formula across the rounds
- Introduce new dependencies, like Bucholz Cut-1
- Change the way data is used : using Burstein "style", pairing players with high rank differences, Dutch "style" with constant rank differences, ...

To build the new weight function $w(i, j, r)$, we can use Deep Learning to create complex formulas that involve a large number of parameters, and fine-tune them to optimize the overall efficiency of the system. As a brief introduction, we start with a very simple linear structure for the weight :

$$w(i, j, r) = x_1.q_1 + ... + x_p.q_p$$

where $q_1, ..., q_p$ are p attributes like the difference of points between the players, or the difference or ranks in the Elo ranking of the scoregroup for example. $x_1, ..., x_p$ are the associated coefficients, giving more or less importance to every attribute. In the previous example, $x_1 = 10000$, $x_2 = 1$, and $p = 2$.

By simulating a lot of tournaments, we can determine the optimal parameters $x_1, ..., x_p$ so that we achieve the highest possible performance. Hence, defining the best possible pairing system will depend on our ability to evaluate every tested set of parameters, our search algorithm that chooses these parameters to test, and the structure of the Neural Network.

The worst tau-b should be 0.7209 and the ideal is 0.8898
Swiss Dutch : 0.7778
Swiss Burstein : 0.8019
Swiss Pheng : 0.8360
Neural Pairings : 0.8025

The worst Euclidean distance should be 1021.0046 and the ideal is 381.2911
Swiss Dutch : 783.7927
Swiss Burstein : 714.7064
Swiss Pheng : 593.4126
Neural Pairings : 704.5419

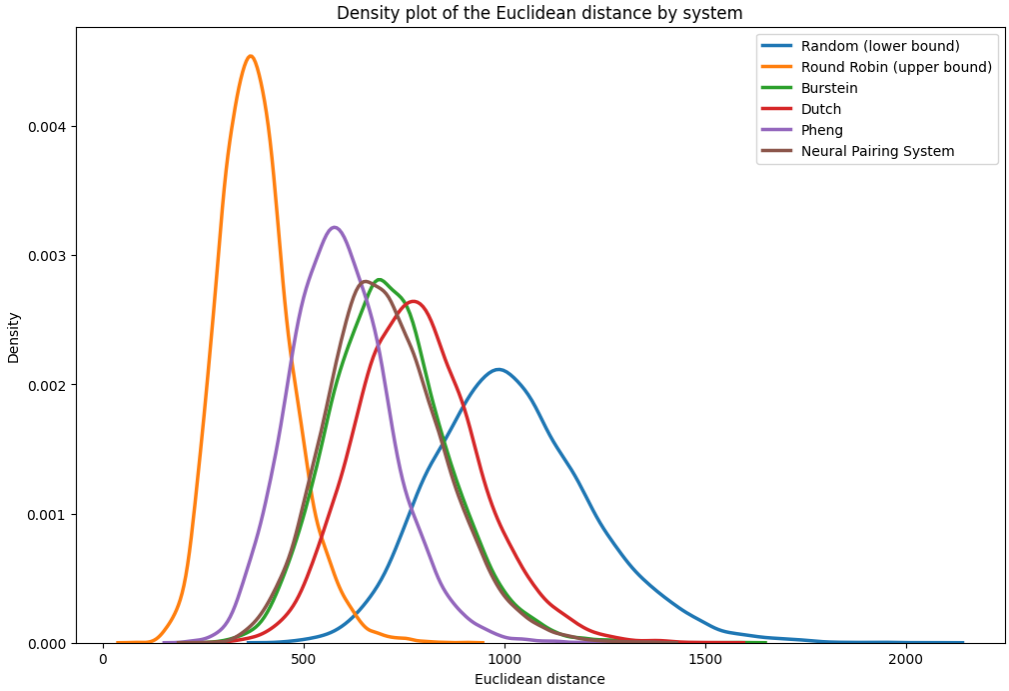


Figure 17: General ranking quality measured by Euclidean distance, values close to 0 means better ranking quality. 10000 simulations with 32 players

The Neural Pairing System in the current state presents little improvement upon the Burstein system for general ranking. However, it shows more promising results for subcategory ranking when fine-tuning for it. Investigating the potential of Neural Networks for choosing pairings through the weights of every edge of our graph modelization can bring new improvements.

11 Conclusion

The Dutch system of pairings is well accepted and remains extremely popular in the chess world, but our simulations confirm the apparent flaws of this format. The Burstein system shows great improvements on all our metrics, but is less used for now as it makes the weakest player play against the strongest. This issue can be solved by changing the system of the first rounds ; as they are the least important ones, it should not have too much negative impact on the rest. The Swiss Random doesn't seem to solve the

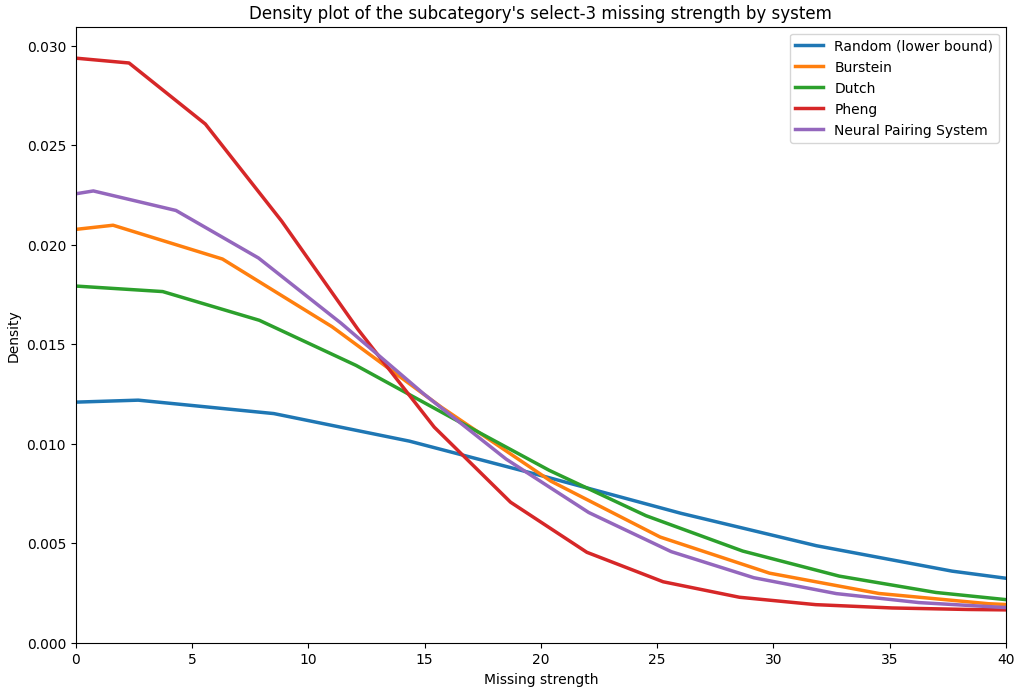


Figure 18: Subcategory ranking quality measured by missing strength, values close to 0 means better ranking quality. 10000 simulations with 32 players

issue and probably does not represent the future of chess tournaments.

Our approach using Machine Learning to determine a better formula for the edges showed promising results but can surely be massively improved by taking into account all the available data with a complex structure for the definition of $w(i, j, r)$. The biggest change lies in the "non-greediness" of this algorithm that behaves differently with respect to the number of remaining rounds, in opposite to the traditional systems that directly aim for pairing people with the closest possible number of points. We can also fine tune it to fulfill the goals of any tournament, whether it qualifies the top k players for a prestigious event or rewards specific categories of players.

The Pheng system we developed showed that the biggest improvement that can easily be done lies is changing the number of points given at each round in order to compensate the importance of the rounds and make players "converge" closer to their theoretical rank. We ran such a tournament at the EPFL Chess Club on May 30th 2024. Players seemed a bit taken aback when

it was announced that the first round was worth 3 pts, but the format was very appreciated. As a player, this rule puts a lot of pressure right from the start. It appeared in the simulations that "flattening" a bit the vector could keep a very good ranking precision, so if a 3-pts first game is too heavy for the players, there are other "nicer" sequences (like $2-2-2-1-1-1-1$) that can be used to satisfy both the quality of the pairings and the players' comfort zone.

References

[J65] Edmonds J. "Paths, Trees, and Flowers". In: *Canadian Journal of Mathematics* (1965). DOI: doi:10.4153/CJM-1965-045-4.

[MW71] D.G. MCVITIE and L. B. WILSON. "The stable marriage problem". In: (1971).

[Irv85] Robert W Irving. "An efficient algorithm for the "stable roommates" problem". In: *Journal of Algorithms* 6 (1985), pp. 577–595. DOI: https://doi.org/10.1016/0196-6774(85)90033-1..

[Óla90] Snjólfur Ólafsson. "Weighted matching in chess tournaments." In: *Journal of the Operational Research Society 41, 1 (1990), 17–24* (1990).

[App95] David R. Appleton. "May the Best Man Win?" In: *Journal of the Royal Statistical Society. Series D (The Statistician)* 44.4 (1995), pp. 529–538. ISSN: 00390526, 14679884. URL: http://www.jstor.org/stable/2348901 (visited on 07/05/2024).

[KLM99] Eija Kujansuu, Tuukka Lindberg, and Erkki Mäkinen. "The Stable Roommates Problem and Chess Tournament Pairings". In: 1999. URL: https://api.semanticscholar.org/CorpusID:16201398.

[Mar05] Shane T. Jensen Mark E. Glickman. "Adaptive paired comparison design," in: *Journal of Statistical Planning and Inference* 127 (2005). DOI: https://doi.org/10.1016/j.jspi.2003.09.022..

[Wei+15] Wei et al. “Swiss-System Based Cascade Ranking for Gait-Based Person Re-Identification”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* (2015). DOI: https://doi.org/10.1609/aaai.v29i1.9454.

[Csa17] László Csató. “On the ranking of a Swiss system chess team tournament”. In: *Annals of Operations Research* 254.1–2 (Feb. 2017), pp. 17–36. ISSN: 1572-9338. DOI: 10.1007/s10479-017-2440-4. URL: http://dx.doi.org/10.1007/s10479-017-2440-4.

[Hua17] Christopher Hua. “The Swiss Tournament Model”. In: (2017).

[Hel20] Mario Held. “SWISS DUBOV AND FIDE SWISS (DUTCH)”. In: (2020).

[Bal22] László Csató Balázs R. Sziklai Péter Biró. “The efficacy of tournament designs”. In: *Computers & Operations Research* 144 (2022). DOI: https://doi.org/10.1016/j.cor.2022.105821..

[23] *Championnat de France Universitaire 2023 Grille américaine après la ronde 9*. http://www.echecs.asso.fr/Resultats.aspx?URL=Tournois/Id/58527/58527&Action=Ga. Accessed: 2024-06-30. 2023.

[CFL23] Ágnes Cseh, Pascal Führlich, and Pascal Lenzner. “The Swiss Gambit”. In: *Adaptive Agents and Multi-Agent Systems*. 2023. URL: https://api.semanticscholar.org/CorpusID:257050619.

[PL23] Ágnes Csehb Pascal Sauer and Pascal Lenzner. “Improving Ranking Quality and Fairness in Swiss-System Chess Tournaments”. In: (2023).

[24a] *max_weight_matching*. Accessed: 2024-07-02. 2024. URL: https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.matching.max_weight_matching.html.

[24b] *OTHER FIDE-APPROVED PAIRING SYSTEMS*. Accessed: 2024-07-01. 2024. URL: https://handbook.fide.com/chapter/OtherApprovedPairingSystems202507.

[24c] *PLAY-OFF AND TIE-BREAK REGULATIONS*. Accessed: 2024-07-04. 2024. URL: https://handbook.fide.com/chapter/TieBreakRegulations042024.

[24d] *Statistics page*. https://www.chessgames.com/chessstats.html. Accessed: 2024-07-04. 2024.

[24e] *weightedtau*. Accessed: 2024-07-02. 2024. URL: https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.weightedtau.html.